Practical Cryptography for Developers

Module 06

Rationale/Topic Outline

- The Cryptographic Toolbox: Symmetric vs. Asymmetric.
- Securing Data at Rest: Python & Java Implementation.
- Securing Data in Motion: SSL/TLS and PKI.
- Digital Signatures & Hashing: Proving Identity and Integrity.
- Key Management: The hardest part of cryptography.

Beyond the Basics

- Encryption is more than just "hiding" text.
- Moving from the "How" (Algorithms) to the "Where" and "When."
- The goal: Ensuring Confidentiality, Integrity, and Authenticity.
- Why developers must use standard libraries instead of "rolling their own" crypto.

The Pillars of Information Security (CIA)

- Confidentiality: Only authorized users can read the data.
- Integrity: Ensuring the data has not been altered.
- Availability: Data is accessible when needed.
- Authenticity: Proving that the sender is who they say they are.

Symmetric Encryption (Secret Key)

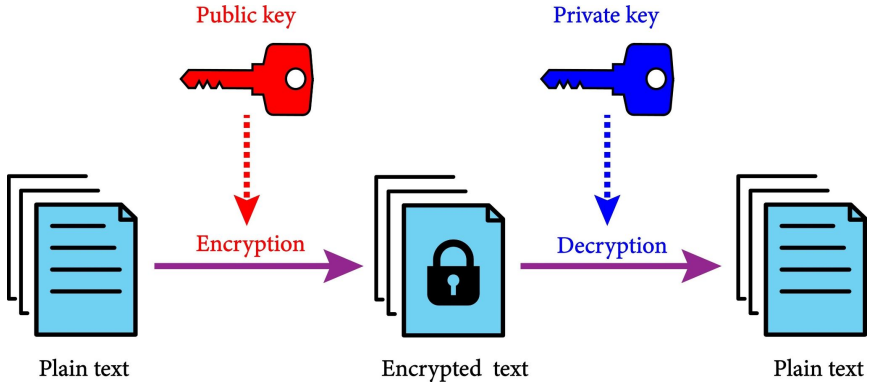
- A single key is used for both encryption and decryption.
- Fast and efficient for large volumes of data.
- The Challenge: How do you share the secret key securely?
- Industry Standard: AES (Advanced Encryption Standard).

Implementing AES in Python

- Using the cryptography library (Fernet).
- Fernet guarantees that a message encrypted cannot be read or altered without the key.
- key = Fernet.generate_key()
- f = Fernet(key); token = f.encrypt(b"Secret Data")

Asymmetric Encryption (Public/Private Key)

- Uses a "Key Pair": A Public Key and a Private Key.
- Public Key: Shared with everyone to encrypt data.
- Private Key: Kept secret by the owner to decrypt data.
- Industry Standard: RSA and Elliptic Curve Cryptography (ECC).



The RSA Workflow

- Bob wants to send a secret to Alice.
- Bob gets Alice's Public Key.
- Bob encrypts the message with it.
- Only Alice's Private Key can unlock it.

Hashing vs. Encryption

- Encryption: A two-way street (Encrypt ↔ Decrypt).
- Hashing: A one-way street (Data ➜ Fingerprint).
- A hash cannot be reversed to find the original input.
- Purpose: Checking if a file has been tampered with.

Secure Hashing Algorithms

- Avoid: MD5 and SHA-1 (They are "broken" and vulnerable).
- Use: SHA-256 or SHA-3.
- For Passwords: Use Argon2 or bcrypt (they include "Salting").
- A "Salt" is random data added to a password before hashing to prevent rainbow table attacks.

Digital Signatures

- Uses Asymmetric keys in reverse.
- The sender "signs" a hash of the message with their Private Key.
- The receiver verifies the signature with the sender’s Public Key.
- Proves: The message is authentic and has not been changed.

Digital Signatures in Java

- Using java.security.Signature class.
- Signature dsa = Signature.getInstance("SHA256withRSA");
- dsa.initSign(privateKey); dsa.update(data);
- byte[] signature = dsa.sign();

Data at Rest (Storage)

- Refers to data stored on a hard drive, database, or cloud.
- Risk: Physical theft of a server or unauthorized database access.
- Solution: Transparent Data Encryption (TDE) or application-level encryption.
- Always encrypt sensitive columns (PII) like SSNs or Credit Cards.

Data in Motion (Transit)

- Refers to data traveling over a network (Internet/WiFi).
- Risk: Man-in-the-Middle (MitM) attacks and packet sniffing.
- Solution: SSL/TLS (HTTPS).
- Cryptography ensures the "tunnel" between the browser and server is private.

The TLS Handshake

- How your browser and server agree on encryption.
  - Client Hello | 2. Server Hello + Certificate.
  - Key Exchange (Asymmetric) | 4. Encrypted Session Starts (Symmetric).
- Why it uses both: Asymmetric for the "handshake," Symmetric for the "speed."

Public Key Infrastructure (PKI)

- A system to manage digital certificates and public keys.
- How do we know a Public Key actually belongs to Google?
- Certificate Authorities (CAs): Trusted third parties that verify identities.
- Examples: Let's Encrypt, DigiCert.

Anatomy of a Digital Certificate

- The Subject (Who the certificate belongs to).
- The Public Key.
- The Digital Signature of the Certificate Authority (CA).
- Expiration Date and Serial Number.

Java Crypto Extension (JCE)

- The framework in Java for cryptographic operations.
- Provides "Providers" like SunJCE or BouncyCastle.
- Handles Ciphers, KeyGenerators, and MACs (Message Authentication Codes).
- Requires proper handling of SecretKeySpec and IvParameterSpec.

Initialization Vectors (IV)

- A random block used to start the encryption process.
- Ensures that encrypting the same text twice results in different ciphertext.
- Rule: Never reuse an IV with the same key.
- The IV does not need to be secret, but it must be unique.

Key Management Challenges

- "Hardcoding keys in source code is a crime."
- If a hacker gets your source code (GitHub), they get your data.
- Solution: Use Environment Variables or Key Vaults (AWS KMS, Azure Key Vault).
- Key Rotation: Periodically changing keys to limit damage if one is leaked.

Common Developer Mistakes

- Using outdated algorithms (DES, Blowfish).
- Insufficient key length (Use 256-bit for AES; 2048-bit+ for RSA).
- Storing keys in the same database as the encrypted data.
- Incorrectly implementing "Homegrown" logic.

General Takeaways

- Symmetric (AES) for speed; Asymmetric (RSA) for key exchange.
- Hashing for integrity; Signatures for authenticity.
- Protect data at rest (Storage) and data in motion (TLS).
- Always use managed libraries and secure your keys.

Thank you